MANUAL OPERACIONAL DE PRODUÇÃO

Guia de Deploy em VPS com Docker & Nginx Proxy Manager

Implantação do Sistema Contador de Apoio ao Tratamento integrado na rede **rede-proxy** com SSL automático.

Rede Docker: rede-proxy **Stack:** React 18 • Node.js 24 • Nginx Alpine • Cloudflare D1 **Ambiente:** VPS Linux / Produção

1. Arquitetura da Solução na VPS

O Nginx Proxy Manager (NPM) é o ponto de entrada único e seguro da VPS. Os contêineres compartilham a mesma rede Docker bridge (`rede-proxy`), permitindo comunicação interna direta por DNS **sem expor portas desnecessárias ao host**.



Vantagem de Segurança: Nenhuma porta da aplicação (como 3000 ou 4000) fica aberta no firewall público da VPS. O tráfego entra exclusivamente pelas portas seguras do proxy reverso.

2. Pré-requisitos na VPS

Requisito	Descrição	Comando de Teste
Docker & Compose	Mecanismo de contêineres instalado na VPS.	`docker --version && docker compose version`
Nginx Proxy Manager	NPM ativo e ouvindo as portas 80 e 443 da VPS.	`docker ps | grep nginx-proxy`
Rede Docker	Rede compartilhada chamada `rede-proxy` .	`docker network ls | grep rede-proxy`
Apontamento DNS	Entrada tipo `A` do domínio/subdomínio apontando para o IP público da VPS.	`dig +short apoio.seudominio.com.br`

3. Roteiro de Implantação Passo a Passo

1 Garantir a Existência da Rede Docker `rede-proxy`

Na VPS, verifique se a rede compartilhada existe. Se não existir, crie-a:

```
# Cria a rede Docker bridge compartilhada:
docker network create rede-proxy

# Se o Nginx Proxy Manager já estiver rodando, conecte-o à rede:
docker network connect rede-proxy <NOME_DO_CONTAINER_NPM>
```

2 Criar o Diretório e Estrutura na VPS

Recomenda-se alocar a aplicação no diretório de serviços padrão (`/opt/contador-apoio`):

```
sudo mkdir -p /opt/contador-apoio
sudo chown -R $USER:$USER /opt/contador-apoio
cd /opt/contador-apoio
```

3 Criar o Arquivo `docker-compose.yml` Refatorado

Crie o arquivo `/opt/contador-apoio/docker-compose.yml` com a versão enxuta:

```
services:
  # 1. API Backend (Node.js 24 + Cloudflare D1 + Google Gemini)
  backend:
    image: maxwellrodriguesferreira/apoio-tratamento:latest
    container_name: contador-apoio-backend
    restart: unless-stopped
    env_file:
      - .env
    volumes:
      - sqlite_data:/app/data
    networks:
      - rede-proxy

  # 2. Frontend Web (React 18 + Nginx SPA)
  frontend:
    build: .
    container_name: contador-apoio-frontend
    restart: unless-stopped
    depends_on:
      - backend
    networks:
      - rede-proxy

volumes:
  sqlite_data:
    name: contador_apoio_d1_data

networks:
  rede-proxy:
    external: true
```

4 Configurar o Arquivo Seguro de Variáveis `.env`

Crie o arquivo `.env` no mesmo diretório (`/opt/contador-apoio/.env`):

```
# === Cloudflare D1 (Banco de Dados Edge SQL em Nuvem) ===
CLOUDFLARE_ACCOUNT_ID=ef579acea3be117e010eafc168ca1c2f
CLOUDFLARE_DATABASE_ID=9e9cb6e6-6981-4357-9c76-acaff75f7377
CLOUDFLARE_API_TOKEN=seu_token_api_cloudflare_aqui

# === Google Gemini API (Análise Executiva e Diagnóstico) ===
GEMINI_API_KEY=sua_chave_do_google_ai_studio_aqui
GEMINI_DEFAULT_MODEL=gemini-3.6-flash

# === Credenciais Iniciais de Administrador ===
ADMIN_EMAIL=admin@seudominio.com.br
ADMIN_PASSWORD=SuaSenhaForteDefinidaAqui123!
```

> **Permissões Restritas:** Execute `chmod 600 .env` para impedir leitura por outros usuários do sistema.

5 Inicializar os Contêineres da Aplicação

Execute o comando abaixo para baixar a imagem da API, compilar o frontend e iniciar os contêineres:

```
docker compose up -d --build
```

Para conferir se os serviços estão operando normalmente:

```
# Status dos contêineres:
docker compose ps

# Acompanhar os logs do backend em tempo real:
docker compose logs -f backend
```

6 Configurar o Host no Nginx Proxy Manager (NPM)

Acesse o painel web do seu NPM (ex: `http://ip-da-vps:81`) e vá em **Hosts ➔ Proxy Hosts ➔ Add Proxy Host**:

Aba "Details" (Detalhes):

Campo	Configuração	Explicação
Domain Names	`apoio.seudominio.com.br`	O subdomínio configurado no DNS para a VPS.
Scheme	`http`	Comunicação interna do Docker é via HTTP simples.
Forward Hostname / IP	`contador-apoio-frontend`	Nome do contêiner Docker na rede `rede-proxy`.
Forward Port	`80`	Porta interna do servidor Nginx do Frontend.
Cache Assets	✅ Ativado	Acelera entrega de arquivos estáticos (CSS, JS).
Block Common Exploits	✅ Ativado	Camada de segurança extra contra bots e ataques.
Websockets Support	✅ Ativado	Garante suporte a conexões em tempo real.

Aba "SSL" (Certificado de Segurança):

Opção	Status	Ação
SSL Certificate	*Request a new SSL Certificate*	Gera certificado Let's Encrypt gratuito e automático.
Force SSL	✅ Ativado	Redireciona todo acesso HTTP (80) para HTTPS (443).
HTTP/2 Support	✅ Ativado	Aumenta a velocidade de carregamento da aplicação.
HSTS Enabled	✅ Ativado	Força os navegadores a usarem sempre conexão segura.

Marque a caixa *"I Agree to the Let's Encrypt Terms of Service"* e clique em **Save**.

4. Resolução de Problemas Frequentes (Troubleshooting)

Sintoma	Causa Provável	Solução
502 Bad Gateway no NPM	O NPM e o frontend não estão na mesma rede Docker.	Execute: `docker network connect rede-proxy <container_npm>` e reinicie o proxy.
Falha ao gerar SSL Let's Encrypt	DNS não propagado ou porta 80 bloqueada no firewall da VPS.	Verifique o DNS com `ping apoio.seudominio.com.br` e certifique-se de que a porta 80 está aberta no firewall (UFW/Security Group).
Erro ao autenticar com o Cloudflare D1	Token Cloudflare sem permissão de edição ou IDs errados.	Confira o `CLOUDFLARE_API_TOKEN` e os logs do backend: `docker compose logs backend`.

5. Comandos Úteis do Dia a Dia

```
# Ver status dos contêineres:
docker compose ps

# Reiniciar os serviços:
docker compose restart

# Visualizar logs em tempo real:
docker compose logs -f

# Atualizar a aplicação para nova versão do código:
git pull && docker compose up -d --build
```

> **Pronto para Uso:** Acesse `https://apoio.seudominio.com.br` para desfrutar da aplicação em ambiente de produção seguro e otimizado!